

# Reproducing Grounding DINO: Open-Vocabulary Object Detection and Visual Grounding on COCO 2017

Su Siqi (12310645), Yang Guanyuhan (12313614), Mo Fengyuan (12311805)  
Southern University of Science and Technology (SUSTech), Shenzhen, China

[https://github.com/YangGuanyuhan/CV\\_Project\\_2026\\_S](https://github.com/YangGuanyuhan/CV_Project_2026_S)

**Abstract**—Open-vocabulary object detection extends traditional detection to arbitrary text-described categories, enabling models to detect objects beyond a fixed training vocabulary. This report presents the reproduction and fine-tuning of Grounding DINO, a state-of-the-art open-vocabulary detector, on the COCO 2017 dataset. Using the official Swin-Tiny backbone checkpoint, we first evaluate the zero-shot baseline, achieving an mAP of 0.485 on the COCO val2017 set. We then fine-tune the model for 6 epochs on the COCO train2017 split, saving a checkpoint after each epoch. The best performing checkpoint (epoch 5) achieves an mAP of 0.559, an AP50 of 0.735, and an AP75 of 0.615, demonstrating significant improvement over the baseline. We also analyze the impact of prompt format design and detection thresholds, and provide a qualitative analysis of success and failure cases. Our modular, config-driven pipeline enables reproducible experimentation and serves as a foundation for further extensions.

**Index Terms**—Open-vocabulary object detection, Grounding DINO, COCO, fine-tuning, vision-language models

## I. INTRODUCTION

Traditional object detection systems are constrained to a fixed set of categories defined during training. Adding new categories requires collecting annotated data and retraining or fine-tuning the model — a process that is both expensive and time-consuming. Open-vocabulary object detection aims to overcome this limitation by detecting objects described by arbitrary text prompts, leveraging the joint understanding of visual and language modalities. This capability has transformative potential across diverse real-world applications: robotic manipulation systems that must interact with novel objects in unstructured environments; medical image analysis for detecting rare pathologies not represented in training data; autonomous driving systems encountering unusual roadside objects; and large-scale image retrieval for e-commerce catalogs with dynamically evolving product taxonomies.

Formally, given an input image  $I \in \mathbb{R}^{H \times W \times 3}$  and a text query  $T$  describing a set of categories  $C_T = \{c_1, c_2, \dots, c_K\}$ , an open-vocabulary detector must produce a set of  $N$  detections  $\{(b_i, c_i, s_i)\}_{i=1}^N$ , where  $b_i = (x_i, y_i, w_i, h_i)$  denotes a bounding box in COCO coordinate format,  $c_i \in C_T$  is the predicted category label, and  $s_i \in [0, 1]$  is the detection confidence score. Unlike traditional detectors where  $C_T$  is a fixed set known at training time, open-vocabulary detection requires the model to generalize to arbitrary  $C_T$  specified at inference time.

This task presents three core challenges rooted in the visual-language interface:

- **Visual-language alignment:** The model must understand both image content and text descriptions, mapping semantic concepts to visual regions. This requires joint embedding spaces where textual descriptions of “person” or “bicycle” activate corresponding visual feature regions.
- **Generalization:** The model must detect categories unseen during training, requiring robust visual-language representations that do not overfit to training category distributions.
- **Fine-grained grounding:** The model must localize specific objects described by natural language phrases with precise bounding boxes. This demands token-level alignment between text phrases and image regions.

This project reproduces the Grounding DINO [1] pipeline and further fine-tunes it on the COCO 2017 training set. Our specific contributions are: (1) a complete zero-shot evaluation on the COCO val2017 benchmark (5,000 images) achieving an mAP of 0.485 using the Swin-Tiny backbone; (2) fine-tuning the model for 6 epochs, with epoch 5 yielding the best performance (mAP=0.559, AP50=0.735, AP75=0.615); (3) an analysis of the prompt format, confirming the critical importance of dot-separated category enumeration; (4) a structured error taxonomy identifying small-object detection and crowded-scene confusion as primary limitations; and (5) an open-source, config-driven reproduction pipeline built on the official `groundingdino-py` package.

The remainder of this report is organized as follows. Section 2 reviews related work on Grounding DINO and open-vocabulary detection. Section 3 describes our reproduction methodology, including model configuration, inference pipeline, and prompt design. Section 4 presents experimental results, including baseline evaluation, fine-tuning results, and ablation studies. Section 5 discusses limitations and future work. Section 6 concludes with key findings.

## II. RELATED WORKS

### A. DETR and DINO: Foundations

Grounding DINO builds upon the DETR (DEtection TRansformer) family of end-to-end object detectors. DETR [2] reformulated object detection as a set prediction problem using a Transformer encoder-decoder architecture with learnable

object queries, eliminating the need for hand-crafted components such as anchor boxes and non-maximum suppression. However, DETR suffered from slow training convergence and difficulty detecting small objects. Deformable DETR [3] addressed these limitations by introducing deformable attention modules that attend to sparse spatial locations around reference points, achieving faster convergence and improved multi-scale feature handling.

DINO [4] (DETR with Improved denoising anchor boxes) further advanced the DETR paradigm through three key innovations: contrastive denoising training, which accelerates convergence by training the model to reconstruct ground-truth objects from noised queries; mixed query selection, which initializes object queries from encoder features rather than learned embeddings; and a look-forward-twice scheme for box prediction refinement. These improvements made DINO competitive with classical detectors while retaining the end-to-end set prediction framework. Grounding DINO [1] extends DINO by fusing text features from a BERT language encoder into the DINO detection architecture, enabling the object queries to become text-conditioned for open-vocabulary detection.

### B. Grounding DINO

Grounding DINO [1] extends the DINO detection architecture with grounded pre-training for open-set detection. The key architectural components include:

- **Swin Transformer backbone** [5]: Provides hierarchical visual features through four stages with decreasing spatial resolution ( $H/4 \times W/4$  to  $H/32 \times W/32$ ) and increasing channel dimension (96 to 768). The shifted-window attention mechanism enables cross-window information exchange while maintaining linear computational complexity with respect to image size.
- **BERT language encoder** [6]: Encodes free-form text prompts into contextualized token embeddings. We use the bert-base-uncased variant (12 Transformer layers, hidden size 768, 12 attention heads) with a maximum text length of 256 tokens.
- **Deformable attention fusion**: Six cross-modal feature fusion layers, each with 8 attention heads and 256-dimensional features, align visual regions with text phrases through deformable attention.

The model uses 900 object queries that interact with both visual features and text features through the fusion layers. Each query predicts a bounding box, a confidence score, and a text-phrase association. The model is pre-trained on diverse detection and grounding datasets including Objects365, OpenImages, and GoldG.

### C. Related Open-Vocabulary Detectors

Open-vocabulary object detection has seen rapid progress since the introduction of vision-language pre-training. Table I compares representative methods.

TABLE I  
COMPARISON OF OPEN-VOCABULARY OBJECT DETECTORS

| Method             | Backbone | Language | Key Innovation                | Year |
|--------------------|----------|----------|-------------------------------|------|
| GLIP [7]           | Swin-T   | BERT     | Unified detection + grounding | 2022 |
| OWL-ViT [8]        | ViT-B/32 | CLIP     | Vision Transformer + CLIP     | 2022 |
| Grounding DINO [1] | Swin-T   | BERT     | DINO + grounded pre-training  | 2024 |
| YOLO-World [9]     | YOLOv8   | CLIP     | Real-time re-parameterization | 2024 |

Among these, Grounding DINO is chosen for our reproduction due to its strong zero-shot performance, well-maintained open-source implementation, and architectural clarity.

### D. COCO Benchmark

Microsoft COCO (Common Objects in Context) [10] is the standard benchmark for object detection evaluation. The dataset contains 80 object categories spanning common everyday objects. The 2017 validation set contains 5,000 images with 36,781 ground-truth bounding box annotations. Evaluation uses the standard COCO metrics: average precision (AP) averaged over 10 IoU thresholds from 0.50 to 0.95, along with per-size breakdowns (APS, APM, APL).

## III. METHOD

We reproduce the Grounding DINO pipeline using the official `groundingdino-py` package [1] and extend it with fine-tuning capabilities. Our work focuses on wrapping the official implementation with a modular Python interface for configurable inference, evaluation, and training.

### A. Model Configuration

Table II summarizes the model specification used in our reproduction.

TABLE II  
COMPLETE GROUNDING DINO MODEL SPECIFICATION

| Component           | Parameter                      | Value                                     |
|---------------------|--------------------------------|-------------------------------------------|
| Swin-T Backbone [5] | window_size / embed_dim        | 7 / 96                                    |
|                     | depths / num_heads             | [2,2,6,2] / [3,6,12,24]                   |
| BERT Encoder [6]    | model / hidden_size            | bert-base-uncased / 768                   |
|                     | max_text_len / vocab_size      | 256 / 30,522                              |
| Deformable Fusion   | d_model / nhead / num_layers   | 256 / 8 / 6                               |
|                     | dim_feedforward                | 1024                                      |
| Detection Head      | num_queries / num_classes      | 900 / 80                                  |
| Input Preprocessing | min_size / max_size            | 800 / 1333                                |
|                     | mean / std (ImageNet)          | [0.485,0.456,0.406] / [0.229,0.224,0.225] |
| Inference           | box_threshold / text_threshold | 0.35 / 0.25 (default)                     |

### B. Inference Pipeline

The inference pipeline consists of four stages: model loading, image preprocessing, text encoding and cross-modal fusion, and postprocessing. Input images are resized such that the shorter side equals 800 pixels while constraining the longer side to at most 1333 pixels. The text prompt uses a dot-separated format enumerating all 80 COCO categories (e.g., “person . bicycle . car . ...”). The model outputs bounding boxes, confidence scores, and phrase indices, which are mapped to COCO category IDs via exact match, substring match, or alias matching.

### C. Fine-Tuning Setup

We fine-tune the pre-trained Grounding DINO Swin-T checkpoint on the COCO train2017 set (118K images) for 6 epochs. The training configuration is as follows:

- Optimizer: AdamW with learning rate  $5 \times 10^{-5}$
- Batch size: 2 (due to GPU memory constraints)
- Mixed precision (AMP) enabled
- Checkpoint saved after every epoch
- Validation performed after every epoch on COCO val2017

We use the same prompt format and threshold settings as in inference. The fine-tuning is implemented using MMDetection and MMEEngine frameworks.

### D. Software Architecture

The reproduction is implemented as a config-driven Python package consisting of five source modules and ten CLI scripts, totaling approximately 1,826 lines of code. Key modules include a model wrapper, predictor, visualizer, COCO evaluator, and dataset utilities. All configuration is managed through OmegaConf YAML files, enabling CLI overrides for paths and hyperparameters.

## IV. EXPERIMENTS

### A. Datasets

We evaluate on the COCO 2017 validation set [10], which contains 5,000 images spanning 80 object categories with 36,781 ground-truth bounding box annotations. For fine-tuning, we use the COCO train2017 set (118K images). All experiments use the standard COCO evaluation metrics.

### B. Implementation Details

All experiments were conducted on an NVIDIA RTX 4090 GPU for training and an RTX 4060 for inference. The software environment includes Python 3.10, PyTorch 2.7.1, CUDA 11.8, and the `groundingdino-py` package (v0.4.0). Default hyperparameters for inference are `box_threshold=0.35` and `text_threshold=0.25`.

### C. Metrics

We use the standard COCO evaluation metrics: AP (averaged over IoU thresholds 0.50:0.95), AP50, AP75, APS, APM, APL, as well as AR@1, AR@10, AR@100 with per-size breakdowns.

### D. Experimental Design

We first evaluate the official pre-trained Grounding DINO Swin-T checkpoint as the baseline (zero-shot). Then, we fine-tune the model on the COCO training set for 6 epochs, saving a checkpoint after each epoch. Each checkpoint is evaluated on the COCO validation set. The goal is to observe whether fine-tuning improves detection performance and to select the best checkpoint for the final demo.

### E. Main Results

Table III presents the complete evaluation results for the baseline and each fine-tuned epoch.

TABLE III  
COCO val2017 EVALUATION RESULTS (BASELINE AND FINE-TUNED CHECKPOINTS)

| Model    | mAP   | AP50  | AP75  | APs   | APm   | APl   |
|----------|-------|-------|-------|-------|-------|-------|
| Baseline | 0.485 | 0.645 | 0.530 | 0.342 | 0.518 | 0.635 |
| Epoch 1  | 0.546 | 0.718 | 0.600 | 0.379 | 0.583 | 0.693 |
| Epoch 2  | 0.553 | 0.722 | 0.606 | 0.400 | 0.592 | 0.701 |
| Epoch 3  | 0.556 | 0.726 | 0.610 | 0.393 | 0.594 | 0.703 |
| Epoch 4  | 0.555 | 0.729 | 0.611 | 0.394 | 0.595 | 0.703 |
| Epoch 5  | 0.559 | 0.735 | 0.615 | 0.400 | 0.593 | 0.712 |
| Epoch 6  | 0.558 | 0.733 | 0.614 | 0.394 | 0.590 | 0.704 |

Epoch 5 achieves the best overall performance (mAP=0.559) and is selected as the final checkpoint for the demo. Fine-tuning significantly improves the model compared with the pretrained baseline: overall mAP improves from 0.485 to 0.559, an absolute gain of 0.074. AP50 improves from 0.645 to 0.735, showing better object recognition and coarse localization. AP75 improves from 0.530 to 0.615, indicating more accurate bounding box localization. The model nearly converges after 5 epochs, as epoch 6 shows a slight drop.

### F. Result Analysis

The per-size metrics reveal that small-object detection (APS) improves from 0.342 to 0.400 after fine-tuning, but remains the primary bottleneck. Large-object detection (APL) reaches 0.712, demonstrating strong performance for bigger objects. The gap between AP50 and AP75 (0.735 vs. 0.615) indicates that localization accuracy still lags behind recognition, especially for small objects.

Qualitative analysis of detection outputs shows that the model successfully detects most medium and large objects across diverse scenes. Common failure modes include:

- **Missed small objects:** Objects smaller than  $32 \times 32$  pixels are frequently undetected, consistent with the APS metric.
- **Crowded-scene confusion:** In dense scenes with overlapping instances, the model may produce duplicate boxes or mislabel occluded objects.
- **Occlusion:** Partial occlusion reduces detection confidence and degrades bounding box precision.

### G. Prompt Format Analysis

We also verify the importance of prompt format. Consistent with the official recommendations, we find that dot-separated prompts (e.g., “person . car . dog .”) are essential for the BERT tokenizer to clearly delineate category boundaries. Using comma-separated or sentence-style prompts leads to a significant drop in performance (approximately 62% AP degradation), confirming the tokenizer-level sensitivity. This finding has direct implications for practical deployment.

## H. Threshold Sensitivity

While we use fixed thresholds (box\_threshold=0.35, text\_threshold=0.25) for evaluation, we observe that threshold selection impacts the precision-recall tradeoff. Lowering the box threshold increases recall but also introduces more false positives. For our fine-tuned model, the optimal threshold may differ; however, for consistency with the official pipeline, we retain the default settings for reporting main results.

## V. DISCUSSION

### A. Comparison with Reported Benchmarks

The official Grounding DINO paper reports a zero-shot AP of approximately 0.485 for the Swin-T backbone, which matches our baseline. After fine-tuning on COCO, the paper reports an AP of approximately 0.506. Our fine-tuned result (AP=0.559) exceeds this reported value, likely due to differences in training hyperparameters, longer training (6 epochs vs. perhaps fewer), or implementation details. This demonstrates that our reproduction is successful and even yields improved performance.

### B. Limitations

The primary limitation remains small-object detection (APS=0.400 even after fine-tuning), attributable to the Swin-T backbone’s resolution constraints. The computational cost of fine-tuning (6 epochs on RTX 4090) is non-negligible, but the resulting model achieves strong performance. The prompt format brittleness is another practical limitation for end-user applications.

### C. Future Work

Future work should explore multi-scale feature fusion, higher input resolution, or alternative backbones to improve small-object detection. Additionally, cross-dataset evaluation and deployment optimization (e.g., TensorRT) are promising directions. The modular pipeline developed in this project facilitates these extensions.

## VI. CONCLUSION

This reproduction successfully demonstrates Grounding DINO’s open-vocabulary detection capability and further improves it via fine-tuning on COCO. Our key findings are:

- The pre-trained model achieves a zero-shot mAP of 0.485 on COCO val2017.
- Fine-tuning for 5 epochs yields the best performance: mAP=0.559, AP50=0.735, AP75=0.615.
- Small-object detection remains the main bottleneck.
- Dot-separated prompts are essential for the model’s text encoder.

The modular, config-driven pipeline provides a solid foundation for future research and deployment. All code and checkpoints are available in our public repository for reproducibility.

## REFERENCES

- [1] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [2] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [3] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [4] H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni, and H.-Y. Shum, “DINO: DETR with improved denoising anchor boxes for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [7] L. H. Li, P. Zhang, H. Zhang, J. Yang, C. Li, Y. Zhong, L. Wang, L. Yuan, L. Zhang, J.-N. Hwang, K.-W. Chang, and J. Gao, “Grounded language-image pre-training,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [8] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, X. Wang, X. Zhai, T. Kipf, and N. Houlsby, “Simple open-vocabulary object detection with vision transformers,” in *European Conference on Computer Vision (ECCV)*, 2022.
- [9] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-world: Real-time open-vocabulary object detection,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014.

## APPENDIX A REPRODUCTION COMMANDS

The following commands reproduce all experiments described in this report. All paths are relative to the repository root.

### A. Environment Setup

```
# Create and activate conda environment
conda create -n grounding_dino python=3.10 -y
conda activate grounding_dino

# Install PyTorch with CUDA 11.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118

# Install core dependencies
pip install groundingdino-py
pip install -r requirements.txt
pip install -e ".[dev]"
```

### B. Data and Model Download

```
# Download Grounding DINO checkpoint (662 MB)
python scripts/download_weights.py

# Download COCO 2017 val2017 and annotations
python scripts/download_coco.py
```

### C. Single-Image Inference

```
python scripts/inference.py \
--image data/demo_images/000000000139.jpg \
--text "person . bicycle . car . motorcycle .
bus ." \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth
```

### D. Full COCO val2017 Evaluation

```
python scripts/eval.py \
--config configs/grounding_dino.yaml \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth \
--coco_image_dir data/coco/val2017 \
--coco_ann_file data/coco/annotations/
instances_val2017.json \
--output_dir outputs/coco_eval/full_val2017
```

### E. Fine-Tuning (6 Epochs)

```
python scripts/train.py \
--config configs/grounding_dino_train.yaml \
--checkpoint checkpoints/groundingdino_swint_ogc
.pth \
--train_ann_file data/coco/annotations/
instances_train2017.json \
--val_ann_file data/coco/annotations/
instances_val2017.json \
--output_dir outputs/finetune
```

## APPENDIX B

### COMPLETE COCO PROMPT

The full dot-separated prompt used for COCO evaluation, enumerating all 80 categories:

```
person . bicycle . car . motorcycle . airplane . bus
. train . truck . boat . traffic light . fire
hydrant . stop sign . parking meter . bench .
bird . cat . dog . horse . sheep . cow .
elephant . bear . zebra . giraffe . backpack .
umbrella . handbag . tie . suitcase . frisbee .
skis . snowboard . sports ball . kite . baseball
bat . baseball glove . skateboard . surfboard .
tennis racket . bottle . wine glass . cup .
fork . knife . spoon . bowl . banana . apple .
sandwich . orange . broccoli . carrot . hot dog
. pizza . donut . cake . chair . couch . potted
plant . bed . dining table . toilet . tv .
laptop . mouse . remote . keyboard . cell phone
. microwave . oven . toaster . sink .
refrigerator . book . clock . vase . scissors .
teddy bear . hair drier . toothbrush .
```

## APPENDIX C

### OUTPUT FILE STRUCTURE

All evaluation and experiment outputs are organized under outputs/:

```
outputs/
  coco_eval/
    full_val2017/      # Main evaluation (5000
images)
  predictions.json     # Detection results in
COCO format
  metrics.json         # Full 12 AP+AR metrics
  eval.log             # Detailed evaluation log
```

```
phrase_mapping.json   # Phrase-to-category
mapping
finetune/             # Fine-tuned checkpoints
and evaluations
  epoch_1/ ... epoch_6/
experiments/
  threshold_sensitivity/ # Optional ablation
  prompt_comparison/    # Optional ablation
visualizations/
  report_highlights/    # Report figures
  coco_eval/           # Success/failure case
images
  inference_demo/       # Single-image inference
examples
inference_results/     # Batch inference outputs
```

## CONTRIBUTIONS

**Su Siqi (12310645):** Contributed to the literature review and background research on open-vocabulary object detection methods. Assisted in the design and execution of the prompt format ablation experiments. Participated in the qualitative error analysis by manually reviewing detection outputs to identify and categorize failure modes.

**Yang Guanyuhan (12313614):** Responsible for the overall project design and implementation. Contributions include: wrapping the official groundingdino-py inference backend with a modular Python interface, building the high-level predictor and visualizer, implementing the COCO evaluator with pycocotools integration, developing dataset utilities for category mapping and prompt construction, and implementing box operation utilities. Created all CLI scripts for inference, evaluation, ablation experiments, and visualization generation. Conducted the full COCO val2017 evaluation and all fine-tuning experiments. Authored the project report.

**Mo Fengyuan (12311805):** Contributed to the design and execution of the threshold sensitivity ablation experiments. Assisted in the quantitative analysis of experimental results, including the computation of per-size AP breakdowns and detection count statistics. Participated in result visualization by helping generate the report highlight figures and organizing the success/failure case grids. Contributed to the writing of the Experiments section and the Discussion section of this report.